

FAMILIARIZATION OF SYSTEM CALLS

02

Aim: Familiarization of system calls (fork, free, getpid, exit, wait, close, stat) in operating system.

System calls used

- fork() → creates a child process
- exec() → replaces process image (runs pg-1)
- getpid() → gets process ID
- wait() → parent waits for child
- close() → close file descriptor
- stat() → get file information
- exit() → terminates process

1. Write the c program OS\_expl

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int fd;
    struct stat fileStat;

    printf("Parent process PID: %d\n", getpid());

    // Open a file
    fd = open("test.txt", O_CREAT | O_RDWR, 0644);
    if (fd < 0) {
        perror("File open failed");
        exit(1);
    }

    // Get file information using stat
    stat("test.txt", &fileStat);
    printf("File size: %ld bytes\n", fileStat.st_size);
}
```



## 3. Expected Output

```

user@user-lp-Laptop-155-eq2xxx:~/Desktop$ gcc OS_exp1.c -o OS_exp1
user@user-lp-Laptop-155-eq2xxx:~/Desktop$ ./OS_exp1
File size: 0 bytes

Child Process
Total 192
-rw-r--r-- 1 user user 104310 Nov  2  2024 ITsertest.pdf
-rwxr-xr-x 1 user user 16432 Jan 28 18:53 05_exp1
-rw-r--r-- 1 user user 1189 Jan 28 18:19 05_exp1.c
-rwxr-xr-x 3 user user 4086 Oct  6  2024 pp
-rwxr-xr-x 4 user user 4086 Feb 13  2025 python
-rwxr-xr-x 5 user user 4086 Sep 29  2024 rahnaa
-rwxr-xr-x 2 user user 4086 Jan 28 18:50 54_IT_OS_LAB
-rwxr-xr-x 2 user user 4086 May  5  2025 testbox
-rw-r--r-- 1 user user 14248 Feb  4  2025 tuple.pdf
-rw-r--r-- 1 user user 18601 Feb  4  2025 'Untitled1.odt'
-rw-r--r-- 1 user user 21 Oct  6  2024 mc

Child process completed

```

```

// Create a new process
pid = fork();

if (pid < 0) {
    perror("Fork failed");
    exit(1);
}

if (pid == 0) {
    // Child process
    printf("Child Process\n");
    printf("Child PID: %d\n", getpid());

    close(fd); // Close file descriptor

    // Replace child with ls command
    exec("ls", "ls", "-l", NULL);

    // exec returns only if it fails
    perror("Exec failed");
    exit(1);
} else {
    // Parent process
    wait(NULL); // Wait for child to finish
    printf("Child process completed\n");
    close(fd);

    return 0;
}

}

2. Compile and Run

gcc OS_exp1.c -o OS_exp1
./OS_exp1

```



## 1.Steps to Partition a Disk using fdisk

### Step 1: Open Terminal

Login as root or use sudo:

```
sudo su
```

### Step 2: List Available Disks

```
fdisk -l
```

→ Displays all disks and existing partitions

Example: /dev/sda, /dev/sdb

### Step 3: Select the Disk to Partition

```
fdisk /dev/sdb
```

(Replace /dev/sdb with your disk name)

### Step 4: Create a New Partition

Inside fdisk prompt, press:

```
n
```

→ Creates a new partition

Choose:

- p → Primary partition
- Partition number (1-4)
- First sector → Press Enter
- Last sector / size → Example +2G

### Step 5: View the Partition Table

```
p
```

→ Displays current partition details

### Step 6: Save the Changes

```
w
```

→ Writes changes to disk and exits fdisk

### Step 7: Format the Partition

```
mkfs.ext4 /dev/sdb1
```

→ Formats partition with ext4 file system

### Step 8: Create a Mount Directory

```
mkdir /mnt/mydisk
```

### Step 9: Mount the Partition

```
mount /dev/sdb1 /mnt/mydisk
```

### Step 10: Verify the Partition

```
df -h
```

→ Shows mounted disk and storage size



## 2.Steps to Install Linux

### Step 1: Download Linux ISO

Download a Linux distribution (example: **Ubuntu**) from the official website.

### Step 2: Create Bootable USB

Use tools like:

- **Rufus** (Windows)
- **Balena Etcher**
- **Startup Disk Creator** (Linux)

Select:

- ISO file
  - USB drive
- Click **Start**

### Step 3: Boot from USB

1. Insert bootable USB
2. Restart the system
3. Press **F2 / F10 / F12 / DEL** to enter BIOS
4. Set USB as first boot device
5. Save and exit

### Step 4: Start Linux Installation

- Choose **Install Linux**
- Select language and keyboard layout

### Step 5: Disk Partitioning

Choose one option:

- **Erase disk and install Linux** (automatic)
- **Manual partitioning** (custom)

Typical partitions:

- **/ (root)** – minimum 20 GB
- **swap** – RAM size



- /home – remaining space (optional)

**Step 6: Configure User Settings**

- Select time zone
- Enter user name and password
- Set computer name

**Step 7: Begin Installation**

Click **Install Now**

Files are copied and system is installed.

**Step 8: Restart System**

After completion:

- Remove installation media
- Click **Restart Now**

**Step 9: Login**

Login using the created username and password.

Result :-

The experiment on familiarization of system calls in an operating system was successfully performed.



Aim : Implementation of Process and thread (Life cycle of Process) :

- i) Process creation & termination
- ii) Thread creation & termination

i) Process creation & termination

A process goes through the following states :-

1. New - Process is being created
2. Ready - Process is waiting to be assigned CPU
3. Running - Process is executing
4. Waiting / Blocked - waiting for I/O or event
5. Terminated - Process has finished execution

- Process creation happens using `fork()`
- Process termination happens using `exit()`

Algorithm :-

1. Start
2. Print "Parent process started"
3. Print "Parent process ID using `getpid()`"
4. Call `fork()` to create a new process
5. If `fork()` returns less than 0
  - Print "Process creation failed"
  - Exit program
6. If `fork()` returns 0 (child process)
  - Print "Child process running"
  - Print child PID



- Print Parent PID
- Print "Child Process Terminated"
- Exit Child Process.

7. Else (Parent process)

- wait for child process using wait()
- Print "Parent Process Resumed"
- Print "Parent Process Terminated"
- Exit Parent Process

8. Stop.

### 1. Write the c program OS\_exp2

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
```

```
int main() {
    pid_t pid;
```

```
    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());
```

```
    // Process creation
    pid = fork();
```

```
    if (pid < 0) {
        printf("Process creation failed\n");
        exit(1);
    }
```

```
    // Child process
    if (pid == 0) {
        printf("\nChild Process Running\n");
        printf("Child PID: %d\n", getpid());
        printf("Child Parent PID: %d\n", getppid());
```

```
        printf("Child Process Terminated\n");
        exit(0); // Child termination
```



### 3.Expected Output

```
Parent Process Started
Parent PID: 9117

Child Process Running
Child PID: 9118
Child Parent PID: 9117
Child Process Terminated

Parent Process Resumed
Parent Process Terminated
```

```
}

// Parent process
else {
    wait(NULL); // Parent waits for child
    printf("\nParent Process Resumed\n");
    printf("Parent Process Terminated\n");
    exit(0); // Parent termination
}
}
```

### 2.Compile and Run

```
gcc OS_exp2.c -o OS_exp2
./OS_exp2
```

### ii)Thread creation and termination

## Life Cycle of a Thread (Short Theory)

A thread passes through the following states:

1. **New** – Thread is created
  2. **Runnable / Ready** – Waiting for CPU
  3. **Running** – Thread is executing
  4. **Blocked / Waiting** – Waiting for resource
  5. **Terminated** – Thread finishes execution
- Thread creation → `pthread_create()`
  - Thread termination → `pthread_exit()`



Algorithm

1. Start
2. Print "Main thread started"
3. Create a new thread using `pthread_create()`
4. Inside thread function:()
  - Print "Thread created & running"
  - Print Thread ID using `pthread_self()`
  - Sleep ~~4000~~ for 2 seconds (simulate work)
  - Print "Thread terminated"
  - Exit thread using `pthread_exit()`
5. In main function:
  - Wait for thread to finish using `pthread_join()`
  - Print "Main thread terminated"
6. Stop.



### 3.Expected Output

```
Main Thread Started
Thread Created and Running
Thread ID: 140568126748224
Thread Terminated
Main Thread Terminated
```

### 1. Write the c program OS\_exp2.1

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Thread function
void* thread_function(void* arg) {
    printf("Thread Created and Running\n");
    printf("Thread ID: %lu\n", pthread_self());

    sleep(2); // Simulate work

    printf("Thread Terminated\n");
    pthread_exit(NULL); // Thread termination
}

int main() {
    pthread_t tid;

    printf("Main Thread Started\n");

    // Thread creation
    pthread_create(&tid, NULL, thread_function, NULL);

    // Wait for thread to finish
    pthread_join(tid, NULL);

    printf("Main Thread Terminated\n");
    return 0;
}
```

### 2.Compile and Run

```
gcc OS_exp2.1.c -o OS_exp2.1
./OS_exp2.1
```

Result :

The experiment on implementation of process and thread (life cycle of process) was successfully performed.



```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    int at[20], bt[20], wt[20], tat[20], ct[20];
```

```
    float avg_wt = 0, avg_tat = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("Enter Arrival Time and Burst Time for P%d: ", i + 1);
```

```
        scanf("%d %d", &at[i], &bt[i]);
```

```
    }
```

```
    ct[0] = at[0] + bt[0];
```

```
    for (i = 1; i < n; i++) {
```

```
        if (ct[i - 1] < at[i])
```

```
            ct[i] = at[i] + bt[i]; // CPU idle case
```

```
        else
```

```
            ct[i] = ct[i - 1] + bt[i];
```

```
    }
```

```
    for (i = 0; i < n; i++) {
```

```
        tat[i] = ct[i] - at[i];
```

```
        wt[i] = tat[i] - bt[i];
```

```
        avg_wt += wt[i];
```

```
        avg_tat += tat[i];
```

```
    }
```

```
    printf("\nProcess\tAT\tBT\tWT\tTAT\n");
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, at[i], bt[i], wt[i], tat[i]);
```

```
    }
```

```
    printf("\nAverage Waiting Time = %.2f", avg_wt / n);
```

```
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat / n);
```

```
    return 0;
```

```
}
```

Date :-

## Process Scheduling Algorithms

12

FCFS, SJF, RR & Priority

Aim : Implement process scheduling algorithms (FCFS, SJF, Round Robin, Priority) & Compute average waiting time & average turn around time.

i) FCFS (First Come First Serve) :

It is the simplest CPU scheduling algorithm.

In this algorithm, the process that arrives first in a ready queue is executed first, just like a queue system.

FCFS which follows non-preemptive scheduling, which means

- Once a process starts execution, it runs until completion.
- CPU cannot be taken away by another process.

### Working Principle

1. Processes enter the ready queue based on their arrival time.
2. CPU selects the process at the front of the queue.
3. After completion, the next process is executed.



Output:

Enter number of processes : 3  
 Enter Arrival time and Burst time for P1 : 0 6  
 Enter Arrival time and Burst time for P2 : 1 7  
 Enter Arrival time and Burst time for P3 : 4 3

| Process | AT | BT | WT | TAT |
|---------|----|----|----|-----|
| P1      | 0  | 6  | 0  | 6   |
| P2      | 1  | 7  | 5  | 12  |
| P3      | 4  | 3  | 9  | 12  |

Average waiting time = 4.67

Average Turnaround Time = 10.00

Algorithm:

1. Start
2. Read the number of processes
3. Read Arrival time and Burst Time for each process
4. Set completion time of first process as Arrival time + Burst Time
5. For each next process, if CPU is idle, set completion time = Arrival time + Burst time; otherwise, add Burst Time to previous completion time.
6. Calculate Turnaround Time = Completion Time - Arrival Time
7. Calculate waiting time = Turnaround Time - Burst Time.
8. Calculate Average waiting Time and Average Turnaround Time
9. Display the ~~process~~ process, AT, BT, WT, TAT and average
10. Stop.